

OPERATING SYSTEMS

(UNIT - IV)

MEMORY MANAGEMENT

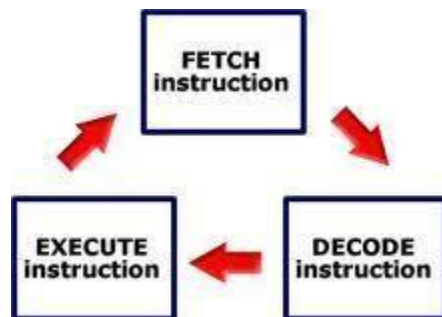
As a result of CPU scheduling, the utilization of the CPU and the speed of the computer's response to the users can be improved. To realize this increase in performance, however, we must keep several processes in memory, that is we must share memory.

For managing this memory, we use various memory management algorithms which vary from a primitive bare machine approach to paging and segmentation strategies. Selection of a memory management method for a specific system depends on many factors, especially on the hardware design of the system.

INTRODUCTION:

Memory is central to the operation of a modern computer system. Memory consists of a large array of bytes or words, each with its own address. The CPU fetches instructions from memory according to the value of the program counter.

A typical instruction execution cycle first fetches an instruction from memory. The instruction is then decoded and may cause operands to be fetched from memory. After the instruction has been executed on the operands, results may be stored back in memory. The memory unit sees only a stream of memory addresses.

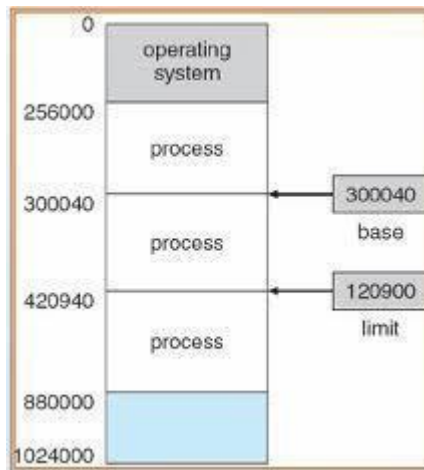


Basic Hardware

Main memory and the registers built into the processor itself are the only storage that the CPU can access directly. Hence any instructions in execution and any data being used by the instructions must be in one of these direct access storage devices. If the data are not in memory, they must be moved there before the CPU can operate on them.

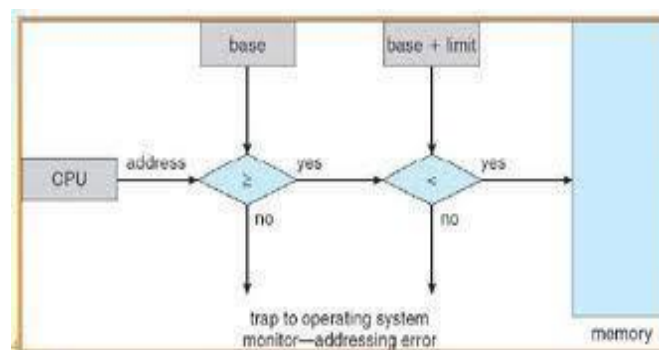
Registers that are built into CPU are generally accessible within one cycle of CPU clock. Main memory is accessed via a transaction on the memory bus. Memory access may take many cycles of the CPU clock to complete, in which case the processor needs to stall since it does not have data required to complete the instruction that it is executing.

Protection of OS from access by user processes and protection of user processes from one another has to be ensured by hardware. One possible implementation is:



Each process should have a separate memory space. For this, a range of legal addresses that the process may access and ensuring that the process accesses only these legal addresses has to be determined. This protection can be provided by using two registers – base and limit register. The base register holds the smallest legal physical memory address and the limit register specifies the size of the range.

Protection of memory space is accomplished by having the CPU hardware compare every address generated in user mode with the registers. Any attempt by a program executing in user mode to access OS memory or other users' memory results in a trap to OS which treats the attempt as a fatal error. This scheme prevents a user program from modifying the code or data structures of either the OS or other users.



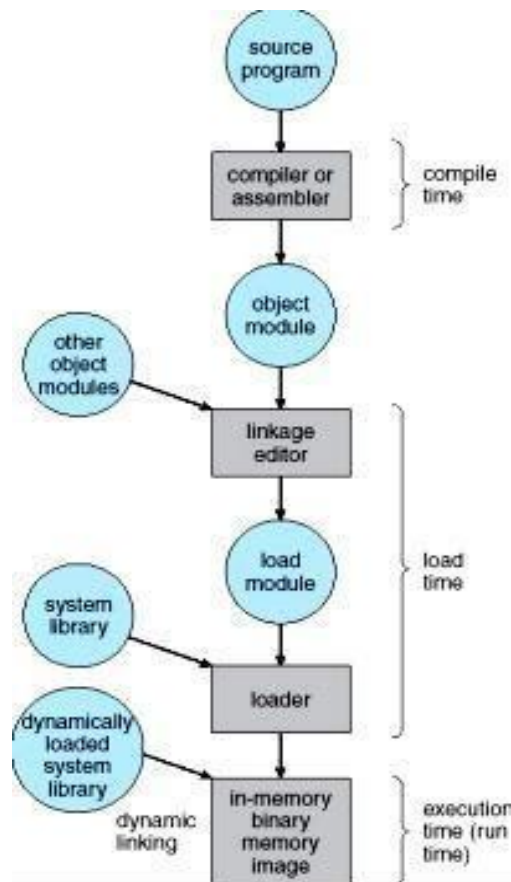
The base and limit registers can be loaded only by the OS which uses a special privileged instruction. Since privileged instructions can be executed only in kernel mode, and since only the OS executes in kernel mode, only the OS can load the base and limit registers. The OS executing in kernel mode is given unrestricted access to both OS and users' memory.

Address Binding

A program resides on a disk as a binary executable file. To be executed, the program must be brought into memory and placed within a process. The processes on the disk that are waiting to be brought into memory for execution form the input queue.

One process from the input queue is selected and loaded into memory. As the process is executed, it accesses instructions and data from memory. Later the process terminates and its memory space is declared available.

A user program goes through the following steps before getting executed. Addresses may be represented in different ways during these steps.



Addresses in the source program are symbolic. A compiler will bind these symbolic addresses to relocatable addresses. The linkage editor or loader will in turn bind the relocatable addresses to absolute addresses. Each binding is a mapping from one address space to another.

The binding of instructions and data to memory addresses can be done at:

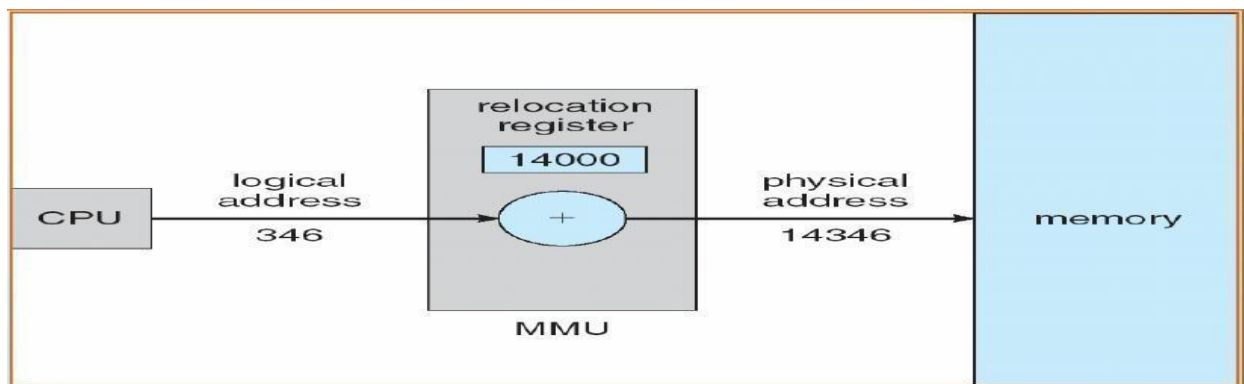
- ✚ Compile time – If you know at compile time, where the process will reside in memory, then absolute code can be generated.
- ✚ Load time – If it is not known at compile time where the process will reside in memory, then the compiler must generate relocatable code. Final binding is delayed until load time.
- ✚ Execution time – If the process can be moved during its execution from one memory segment to another, then binding must be delayed until run time.

Logical versus Physical Address Space

An address generated by the CPU is referred to as logical address or virtual address, whereas an address seen by the memory unit is referred to as physical address.

The set of all logical addresses generated by a program is called logical address space; the set of all physical addresses generated by a program is called physical address space. Thus, in the execution time address binding scheme, the logical and physical address spaces differ.

The run time mapping from virtual to physical addresses is done by a hardware device called the memory management unit (MMU).



The base register here is called a relocation register. The value in the relocation register is added to the logical address to get the equivalent physical address, which is used for accessing the main memory.

Dynamic Loading

The entire program and all data of a process must be in physical memory for the process to execute. The size of a process is limited to the size of physical memory. To obtain memory space utilization, dynamic loading can be used.

With dynamic loading, a routine is not loaded until it is called. All routines are kept on disk in a relocatable load format. The main program is loaded into memory and is executed. When a routine needs to call another routine, the calling routine first checks to see whether the other routine has been loaded. If not, the relocatable linking loader is called to load the desired routine into memory and to update the program's address tables to reflect this change. Then control is passed to the newly loaded routine.

The advantage of dynamic loading is that an unused routine is never loaded. This method is useful when large amounts of code are needed to handle infrequently occurring cases such as error routines. Dynamic loading does not require special support from the OS.

Dynamic linking and shared libraries

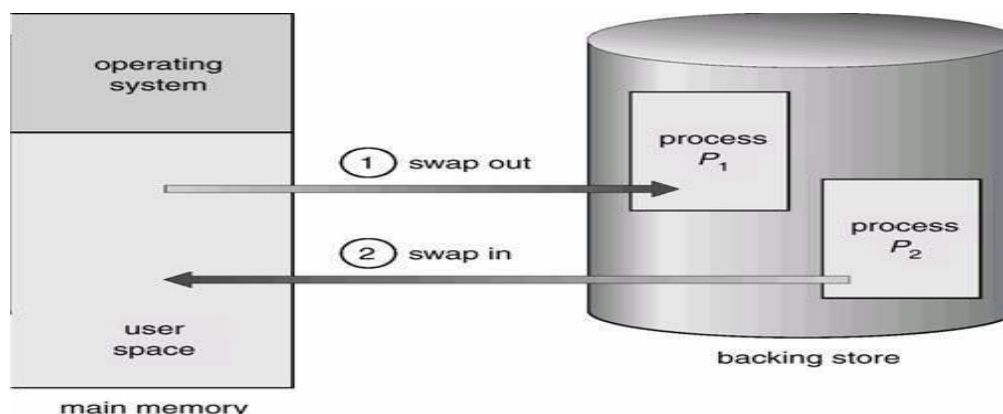
Some OS support only **static linking**, in which system language libraries are treated like any other object module and are combined by the loader into the binary program image. Dynamic linking is done at execution time. This is used with system libraries such as language subroutine libraries. Without this facility, each program on a system must include a copy of its language library in the executable image which wastes both disk space and main memory.

This feature can be extended to library updates. A library may be replaced by a new version and all programs that reference the library will automatically use the new version. This is known as **shared libraries**.

Dynamic linking requires help from the OS. If the processes in memory are protected from one another, then the OS is the only entity that can check to see whether the needed routine is in another process's memory space or that can allow multiple processes to access the same memory addresses.

SWAPPING:

A process must be in memory to be executed. A process can however be swapped temporarily out of memory to a backing store and then brought back into memory for continued execution.



For example: round robin scheduling algorithm. When a time quantum expires, the memory manager will start to swap out the process that just finished and to swap another process into memory. In the meantime, the CPU scheduler will allocate a time slice to some other process. When each process finishes its time, it will be swapped with another process.

A variant of the swapping policy is used for priority based scheduling algorithms. If a higher priority process arrives and wants service, the memory manager can swap out the lower priority process and then load and execute the higher priority process. When the higher priority process finishes, the lower priority process can be swapped back in and continued. This variant of swapping is called **roll out, roll in**.

Swapping requires a backing store. This has to be a fast disk, large enough to accommodate copies of all memory images for all users, and must provide direct access to these memory images. The system maintains a **ready queue** consisting of all processes whose memory images are on the backing store or in memory and are ready to run. Whenever the CPU scheduler decides to execute a process, it calls the dispatcher. The dispatcher checks to see whether the next process in the queue is in memory. If it is not, and if there is no free memory region, the

dispatcher swaps out a process currently in memory and swaps in the desired process. It then reloads registers and transfers control to the selected process.

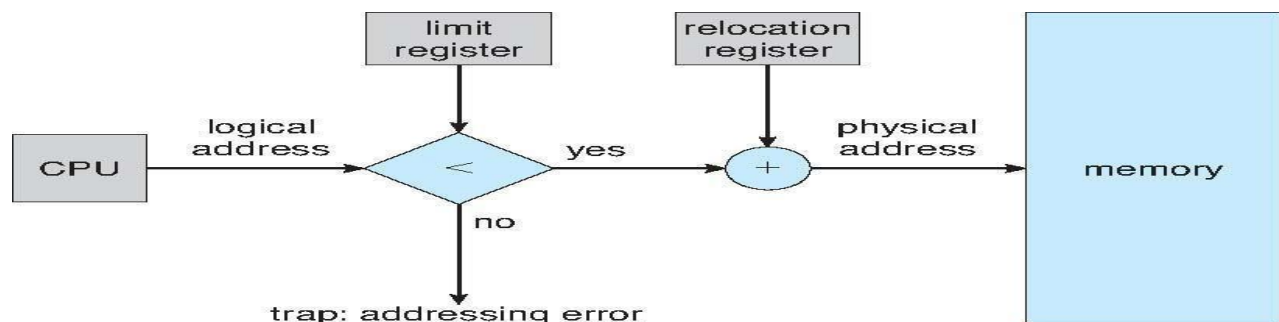
CONTIGUOUS MEMORY ALLOCATION:

The main memory must accommodate both the OS and various user processes. The memory is usually divided into two partitions: one for the OS and one for the user processes. OS can be placed either in low memory or high memory. The major factor affecting this decision is the location of the interrupt vector.

Several user processes should reside in memory at the same time. In contiguous memory allocation, each process is contained in a single contiguous section of memory.

Memory Mapping and Protection

Memory mapping and protection are provided by using relocation register with a limit register. The relocation register contains the value of the smallest physical address; the limit register contains the range of logical addresses. With relocation and limit registers, each logical address must be less than the limit register; the MMU maps the logical address dynamically by adding the value in the relocation register. This mapped address is sent to the memory.



When the CPU scheduler selects a process for execution, the dispatcher loads the relocation and limit registers with the correct values as part of the context switch. Because every address generated by the CPU is checked against these registers, both the OS and other users' programs and data can be protected from being modified by the running process. The relocation register scheme provides an effective way to allow the OS size to change dynamically.

Memory Allocation

Fixed sized partitions:

One of the simplest methods for allocating memory is to divide memory into several fixed – sized partitions. Each partition may contain exactly one process. Thus, the degree of multi programming is bound by the number of partitions. In this multiple partition method, when a partition is free, a process is selected from the input queue and is loaded into the free partition. When the process terminates, the partition becomes available for another process.

In the fixed partition scheme, the OS keeps a table indicating which parts of memory are available and which are occupied. Initially, all memory is available for user processes and is considered one large block of available memory called a hole. When a process arrives and needs memory, we search for a hole large enough for this process.

As processes enter the system, they are put into an input queue. The OS takes into account the memory requirements of each process and the amount of available memory space in determining which processes are allocated memory. When a process is allocated space, it is loaded into memory, and then competes for CPU. When a process terminates, it releases its memory, which the OS may then fill with another process from the input queue.

variable sized partitions:

At any given time, we have a set of holes of various sizes scattered throughout memory. When a process arrives and needs memory, the system searches the set for a hole that is large enough for this process. If the hole is too large, it is split into two parts. One part is allocated to the arriving process and the other is returned to the set of holes. When a process terminates, it releases its block of memory which is then placed back in the set of holes. If the new hole is adjacent to other holes, these adjacent holes are merged to form one larger hole. Here the system checks whether there are processes waiting for memory and whether this newly freed and recombined memory could satisfy the demands of any of these waiting processes.

This procedure is an instance of the general dynamic storage allocation problem. The solutions to this problem are –

- a) First fit – allocate the first hole that is big enough
- b) Best fit – allocate the smallest hole that is big enough
- c) Worst fit – allocate the largest hole.

Fragmentation

Both the first fit and best fit strategies for memory allocation suffer from **external fragmentation**. As processes are loaded and removed from memory, the free memory space is broken into little pieces.

External fragmentation:

External fragmentation exists when there is enough total memory space to satisfy a request but the available spaces are not contiguous.

Depending on the total amount of memory storage and the average process size, external fragmentation can be a minor or major problem. Statistical analysis of first fit reveals that even with some optimization, given N allocated blocks, another $0.5 N$ blocks will be lost to fragmentation. This property is known as the **50 percent rule**.

Internal fragmentation:

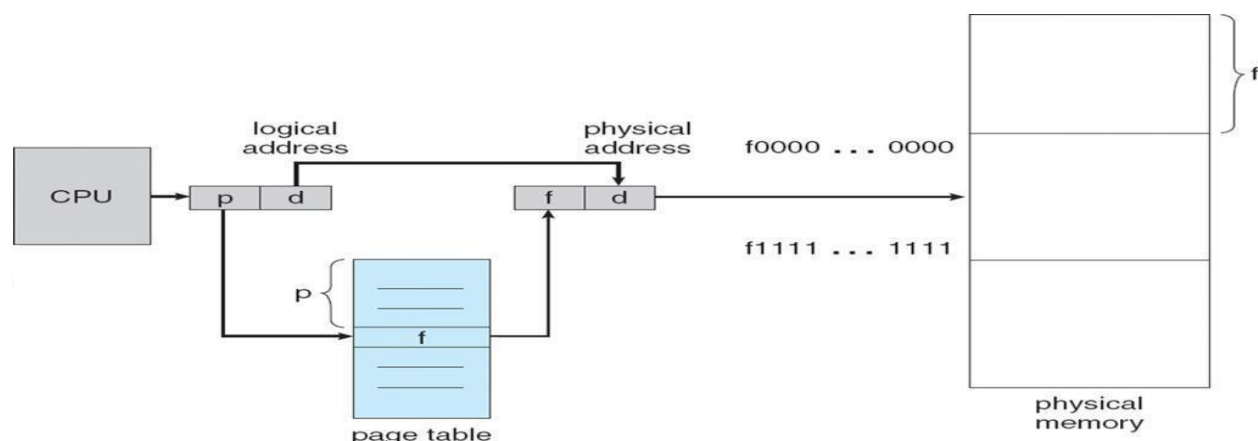
Memory fragmentation can be internal as well as external. When the physical memory is broken down into fixed sized blocks and memory is allocated in units based on block size, the memory allocated to a process may be slightly larger than the requested memory. The difference between these two numbers is **internal fragmentation** – memory that is internal to a partition but is not being used.

One solution to the problem of external fragmentation is **compaction**. Here the goal is to shuffle the memory contents so as to place all free memory together in one large block. If relocation is static and is done at assembly or load time, compaction cannot be done; compaction is possible only if relocation is dynamic and is done at execution time

Another possible solution to the external fragmentation problem is to permit the logical address space of the processes to be non contiguous; thus allowing a process to be allocated physical memory wherever the latter is available. Two techniques achieve this solution – paging and segmentation.

PAGING:

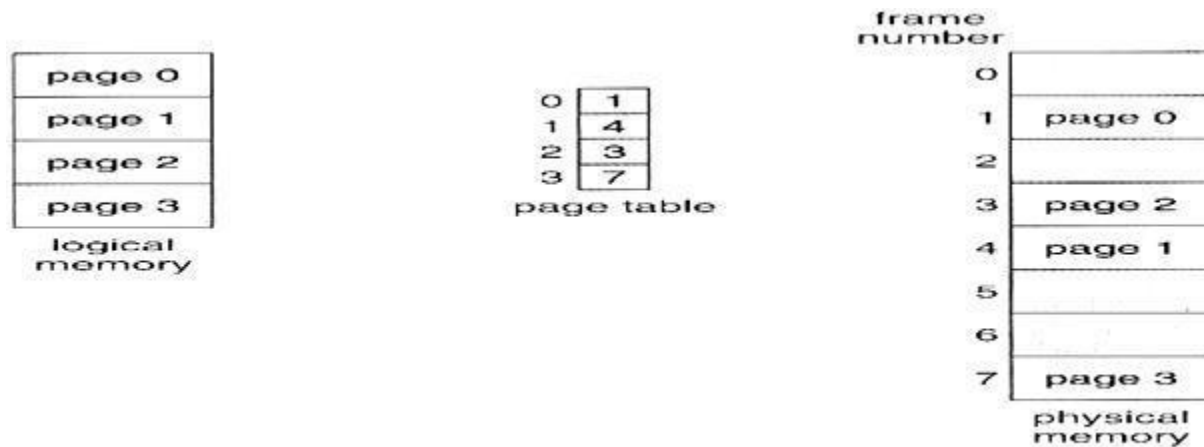
Paging is a memory management scheme that permits the physical address space of a process to be non contiguous. Paging avoids the problem of fitting memory chunks of varying sizes onto the backing store. Support for paging is handled by hardware.



Basic Method

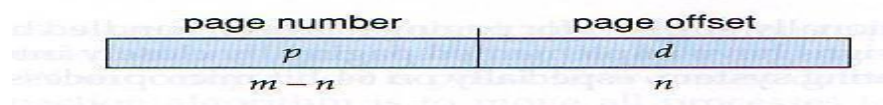
The basic method for implementing paging involves breaking physical memory into fixed sized blocks called **frames** and breaking logical memory into blocks of the same size called **pages**. When a process is to be executed, its pages are loaded into any available memory frames from the backing store. This backing store is divided into fixed sized blocks that are of the same size as the memory frames.

Every address generated by the CPU is divided into two parts: a **page number (p)** and a **page offset (d)**. The page number is used as an index into a page table. The page table contains the base address of each page in physical memory. This base address is combined with the page offset to define the physical memory address that is sent to the memory unit.



Paging model of logical and physical memory.

If the size of the logical address space is 2^m , and the page size is 2^n addressing units then the high order $m-n$ bits of a logical address designate the page number and the n low order bits designate the page offset.



Logical address is given as above where p is an index into the page table and d is the displacement within the page.

Paging itself is a form of dynamic relocation. Every logical address is bound by the paging hardware to some physical address. When paging scheme is used, there is no external fragmentation.

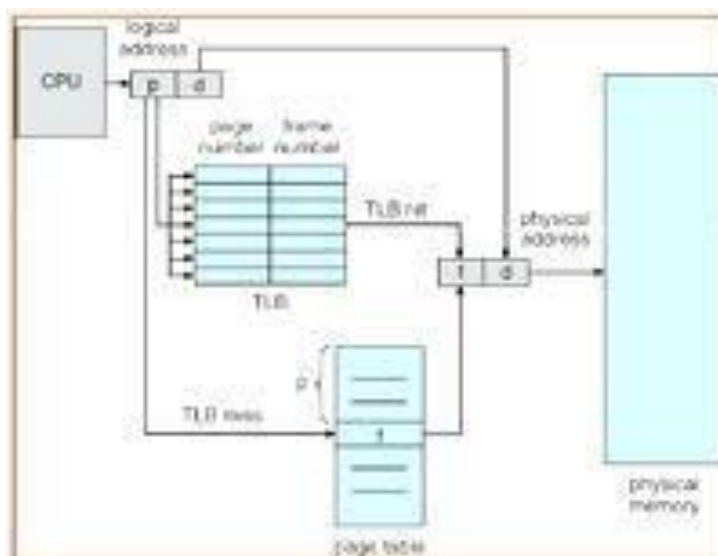
Hardware Support

Most OS's allocate a page table for each process. A pointer to the page table is stored with the other register values in the process control block. When the dispatcher is told to start a process, it must reload the user registers and define the correct hardware page table values from the stored user page table.

In the hardware implementation of the page table, it can be implemented as a set of dedicated **registers**. These registers should be built with very high speed logic to make the paging address translation efficient. Every access to memory must go through the paging map. The use of registers is satisfactory if the page table is small. For contemporary computers, where page table is very large, the use of fast registers to implement the page table is not feasible. Hence the page table is kept in main memory and a **page table base register (PTBR)** points to the page table. Changing the page tables requires changing only this one register reducing context switch time.

But the time required for accessing the user location is too large. The solution to this problem is to use a special small fast lookup hardware cache called translation look aside buffer (TLB). The TLB is associative, high speed memory. Each entry in the TLB consists of two parts – a key(or a tag) and a value.

The TLB contains only a few of the page table entries. When a logical address is generated by the CPU its page number is presented to the TLB. If the page number is found, its frame number is immediately available and is used to access memory. If the page number is not in the TLB known as TLB miss, a memory reference to the page table must be made. When the frame number is obtained, it can be used to access memory. And then, we add the page number and frame number to the TLB, so that they will be used in the next reference. If the TLB is already full of size, the OS must select one for replacement.



Protection

Refer fig no: 8.12 Page no: 285

Memory protection in a paged environment is accomplished by protection bits associated with each frame. These bits are kept in the page table.

One additional bit is generally attached to each entry in the page table – a **valid – invalid bit**.

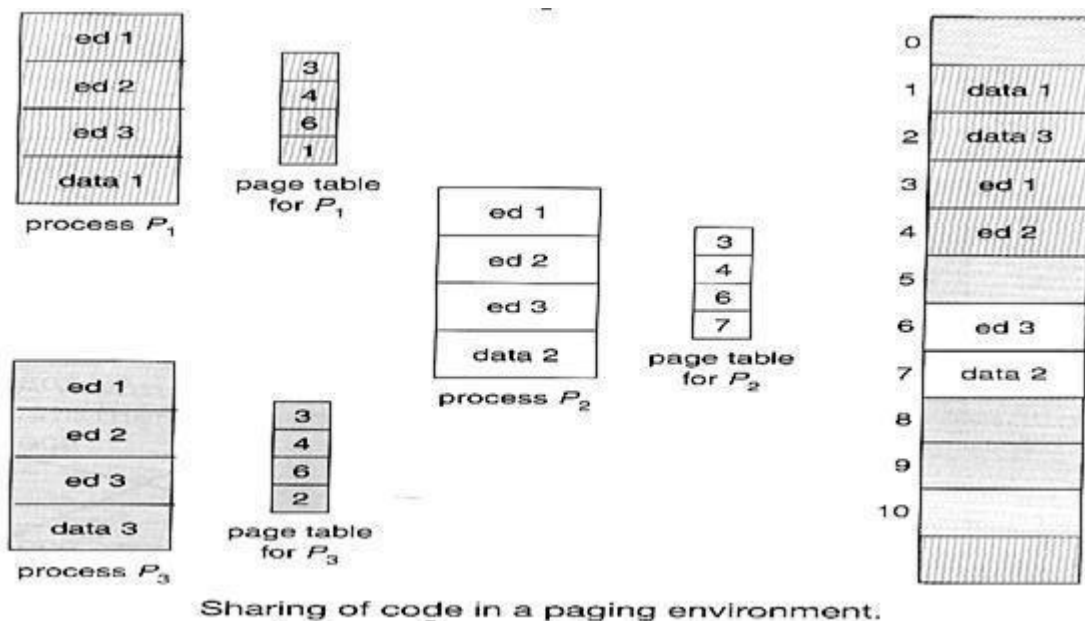
When this bit is set to **valid**, the associated page is in the process's logical address space and is thus a legal page. When the bit is set to **invalid**, the page is not in the process's logical address space. OS sets this bit for each page to allow or disallow access to the page.

Some systems provide hardware in the form of a **page table length register (PTLR)** to indicate the size of the page table. This value is checked against every logical address to verify that the address is in the valid range for the process. Failure of this test causes an error trap to the OS.

Shared Pages

An advantage of paging is the possibility of sharing common code. This is important in a time sharing environment.

If the code is **reentrant code (or pure code)**; it can be shared.



Reentrant code is non self modifying code; it never changes during execution. Thus, two or more processes can execute the same code at the same time. Each process has its own copy of registers and data storage to hold the data for the process's execution. The data for different processes will be different.

Only one copy of the editor need to be kept in physical memory. Each user's page table maps onto the same physical copy of the editor, but data pages are mapped onto different frames. Other heavily used programs such as compilers, window systems, run – time libraries, database systems can also be shared. To be sharable, the code must be reentrant.

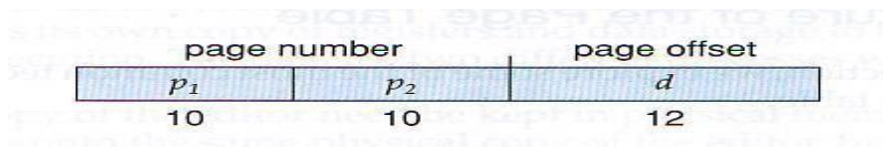
STRUCTURE OF THE PAGE TABLE:

Common techniques for structuring page table –

Hierarchical Paging

Most modern computer systems support a large logical address space. In such an environment, the page table itself becomes excessively large. For example: consider a system with a 32-bit logical address space. If the page size is 2^{12} , then a page table consists of $2^{32} / 2^{12}$. So, there are max. number of entries in the page table. One solution to this problem is to divide the page table into smaller pieces. This can be accomplished in several ways.

One way is to use a two level paging algorithm in which the page table itself is also paged.



In the above example, a logical address is divided into a page number consisting of 20 bits and a page offset consisting of 12 bits. As the page table is paged, page number is further divided into a 10 bit page number and a 10 bit page offset. In the above figure, p_1 is an index into the outer page table, p_2 is the displacement within the page of the outer page table. Because address translation works from the outer page table inward, this scheme is also known as a **forward mapped page table**.

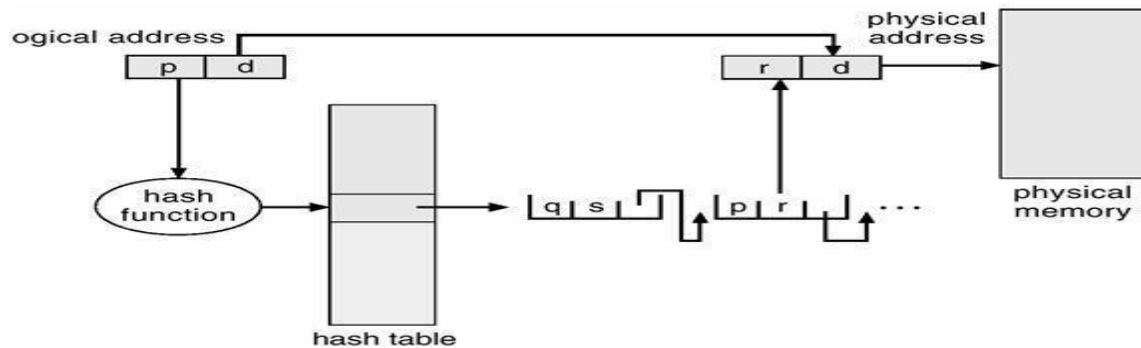
Hashed Page Tables

A common approach for handling address spaces larger than 32 bits is to use a **hashed page table** with the hash value being the virtual page number. Each entry in the hash table contains a linked list of elements that hash to the same location. Each element consists of three fields:

1. The virtual page number
2. The value of the mapped page frame
3. Pointer to the next element in the linked list

It works as follows –

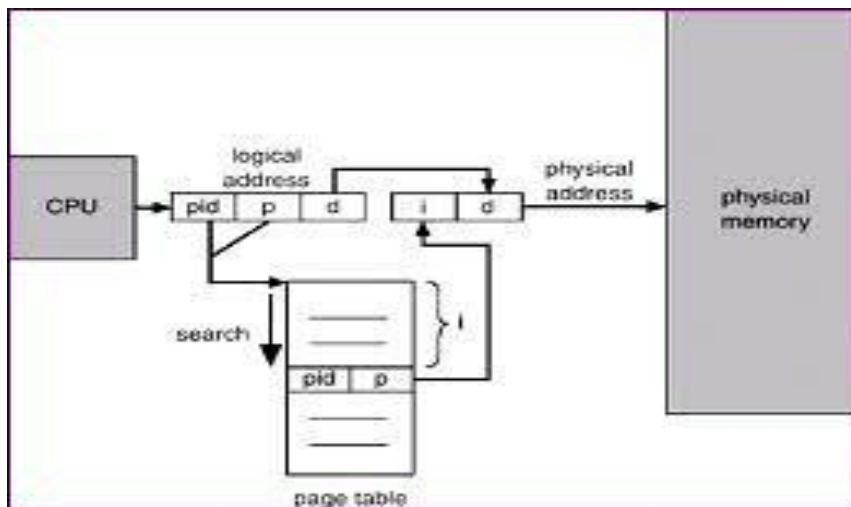
The virtual page number in the virtual address is hashed into the hash table. The virtual page number is compared with field1 in the first element in the linked list. If there is a match, the corresponding page frame (field2) is used to form the desired physical address. If there is no match, subsequent entries in the linked list are searched for a matching virtual page number.



Inverted Page Tables

Each process has an associated page table. The page table has one entry for each page that the process is using. Processes reference pages through the pages' virtual addresses. The OS must then translate this reference into a physical memory address. Since the table is sorted by virtual address, the OS can calculate where in the table the associated physical address entry is and to use that value directly. One of the drawbacks of this method is that each page table may consist of millions of entries.

To solve this problem, we use an inverted page table.



An inverted page table has one entry for each real page (or frame) of memory. Each entry consists of the virtual address of the page stored in that real memory location; with information about the process that owns that page. Thus, only one page table is in the system and it has only one entry for each page of physical memory.

Each virtual address in the system consists of a triple:

< process-id, page number, offset >

Each inverted page table entry is a pair < process-id, page number > . when a memory reference occurs, part of the virtual address, consisting of < process-id, page number > is presented to the memory sub system. The inverted page table is then searched for a match. If a match is found- say at i- then the physical address <i,offset> is generated. If no match is found, then an illegal address access has been attempted.

VIRTUAL MEMORY

Virtual Memory is a space where large programs can store themselves in form of pages while their execution and only the required pages or portions of processes are loaded into the main memory. This technique is useful as a large virtual memory is provided for user programs when a very small physical memory is there. Thus Virtual memory is a technique that allows the execution of processes that are not in the physical memory completely.

Virtual Memory mainly gives the illusion of more physical memory than there really is with the help of Demand Paging.

In real scenarios, most processes never need all their pages at once, for the following reasons:

- Error handling code is not needed unless that specific error occurs, some of which are quite rare.
- Arrays are often over-sized for worst-case scenarios, and only a small fraction of the arrays are actually used in practice.
- Certain features of certain programs are rarely used.

In an Operating system, the memory is usually stored in the form of units that are known as **pages**. Basically, these are atomic units used to store large programs.

Virtual memory can be implemented with the help of:-

1. Demand Paging
2. Demand Segmentation

Need of Virtual Memory

Following are the reasons due to which there is a need for Virtual Memory:

- In case, if a computer running the Windows operating system needs more memory or RAM than the memory installed in the system then it uses a small portion of the hard drive for this purpose.
- Suppose there is a situation when your computer does not have space in the physical memory, then it writes things that it needs to remember into the hard disk in a swap file and that as virtual memory.

Benefits of having Virtual Memory

1. Large programs can be written, as the virtual space available is huge compared to physical memory.
2. Less I/O required leads to faster and easy swapping of processes.
3. More physical memory available, as programs are stored on virtual memory, so

they occupy very little space on actual physical memory.

4. Therefore, the Logical address space can be much larger than that of physical address space.
5. Virtual memory allows address spaces to be shared by several processes.
6. During the process of creation, virtual memory allows: **copy-on-write** and **Memory-mapped files**.

DEMAND PAGING:

Every process in the virtual memory contains lots of pages and in some cases, it might not be efficient to swap all the pages for the process at once. Because it might be possible that the program may need only a certain page for the application to run. Let us take an example here, suppose there is a 500 MB application and it may need as little as 100MB pages to be swapped, so in this case, there is no need to swap all pages at once.

The demand paging system is somehow similar to the paging system with swapping where processes mainly reside in the main memory (usually in the hard disk). Thus demand paging is the process that solves the above problem only by swapping the pages on Demand. This is also known as **lazy swapper** (It never swaps the page into the memory unless it is needed).

Swapper that deals with the individual pages of a process are referred to as **Pager**.

Demand Paging is a technique in which a page is usually brought into the main memory only when it is needed or demanded by the CPU. Initially, only those pages are loaded that are required by the process immediately. Those pages that are never accessed are thus never loaded into the physical memory.



Figure: Transfer of a Paged Memory to the contiguous disk space.

Whenever a page is needed? make a reference to it;

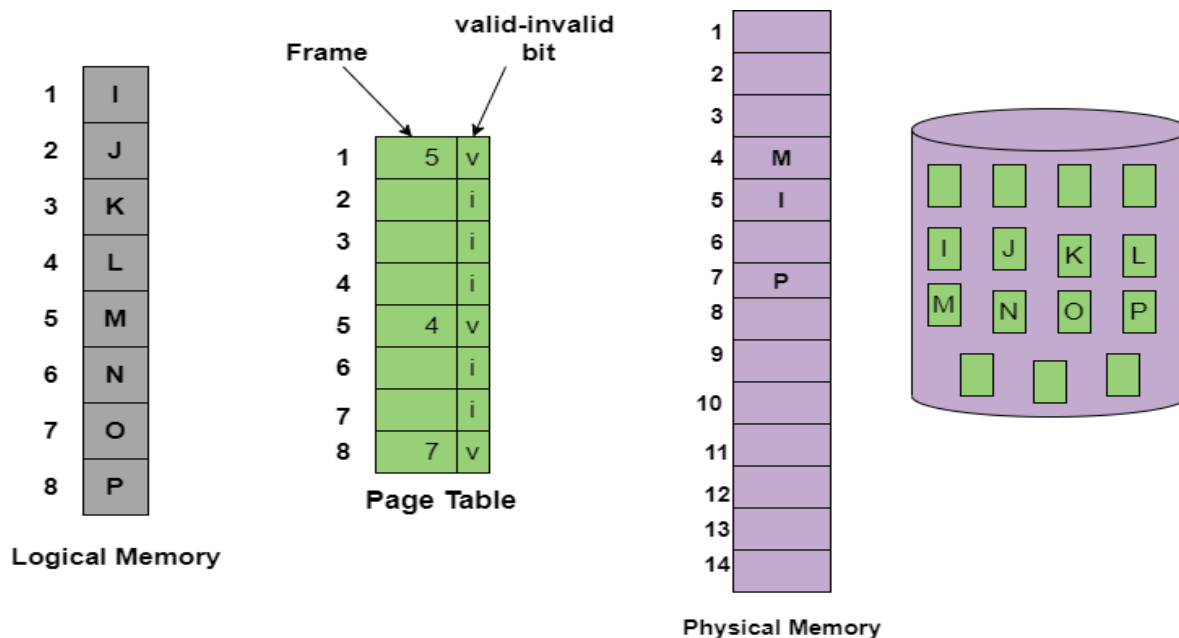
- If the reference is **invalid** then abort it.
- If the page is Not-in-memory then bring it to memory.

Valid-Invalid Bit:

Some form of hardware support is used to distinguish between the pages that are in the memory and the pages that are on the disk. Thus for this purpose Valid-Invalid scheme is used:

- With each page table entry, a valid-invalid bit is associated(where **1** indicates in **The memory** and **0** indicates **not in the memory**)
 - Initially, the valid-invalid bit is set to 0 for all table entries.
1. If the bit is set to "**valid**", then the associated page is **both legal and is in memory**.
 2. If the bit is set to "**invalid**" then it indicates that the **page is either not valid** or the **page is valid but is currently not on the disk**.
- For the **pages** that are **brought into the memory**, the **page table** is set as usual.
 - But for the **pages** that are **not currently in the memory**, the **page table** is either **simply marked as invalid** or it contains the **address of the page on the disk**.

During the translation of address, if the valid-invalid bit in the page table entry is 0 then it leads to **page fault**.



How Demand Paging Works?

First of all the components that are involved in the Demand paging process are as follows:

- Main Memory
- CPU
- Secondary Memory

- Interrupt
 - Physical Address space
 - Logical Address space
 - Operating System
 - Page Table
1. If a page is not available in the main memory in its active state; then a request may be made to the CPU for that page. Thus for this purpose, it has to generate an interrupt.
 2. After that, the Operating system moves the process to the blocked state as an interrupt has occurred.
 3. Then after this, the Operating system searches the given page in the Logical address space.
 4. And Finally with the help of the page replacement algorithms, replacements are made in the physical address space. Page tables are updated simultaneously.
 5. After that, the CPU is informed about that update and then asked to go ahead with the execution and the process gets back into its ready state.

When the process requires any of the pages that are not loaded into the memory, a page fault trap is triggered and the following steps are followed,

1. The memory address which is requested by the process is first checked, to verify the request made by the process.
2. If it is found to be invalid, the process is terminated.
3. In case the request by the process is valid, a free frame is located, possibly from a free-frame list, where the required page will be moved.
4. A new operation is scheduled to move the necessary page from the disk to the specified memory location. (This will usually block the process on an I/O wait, allowing some other process to use the CPU in the meantime.)
5. When the I/O operation is complete, the process's page table is updated with the new frame number, and the invalid bit is changed to valid.
6. The instruction that caused the page fault must now be restarted from the beginning.

Advantages of Demand Paging

The benefits of using the Demand Paging technique are as follows:

- With the help of Demand Paging, memory is utilized efficiently.
 - Demand paging avoids External Fragmentation.
 - Less Input/Output is needed for Demand Paging.
 - This process is not constrained by the size of physical memory.
 - With Demand Paging it becomes easier to share the pages.
 - With this technique, portions of the process that are never called are never loaded.
 - No compaction is required in demand Paging.

Disadvantages of Demand paging

Drawbacks of Demand Paging are as follows:

- There is an increase in overheads due to interrupts and page tables.
- Memory access time in demand

paging is longer.**Pure Demand Paging**

In some cases when initially no pages are loaded into the memory, pages in such cases are only loaded when are demanded by the process by generating page faults. It is then referred to as **Pure Demand Paging**.

- In the case of pure demand paging, there is not even a single page that is loaded into the memory initially. Thus pure demand paging causes the page fault.
- When the execution of the process starts with no pages in the memory, then the operating system sets the instruction pointer to the first instruction of the process and that is on a non-memory resident page and then in this case the process immediately faults for the page.
- After that when this page is brought into the memory then the process continues its execution, page fault is necessary until every page that it needs is in the memory.
- And at this point, it can execute with no more faults.
- This scheme is referred to as Pure Demand Paging: means never bring a page into the memory until it is required.

Handling the Page Fault

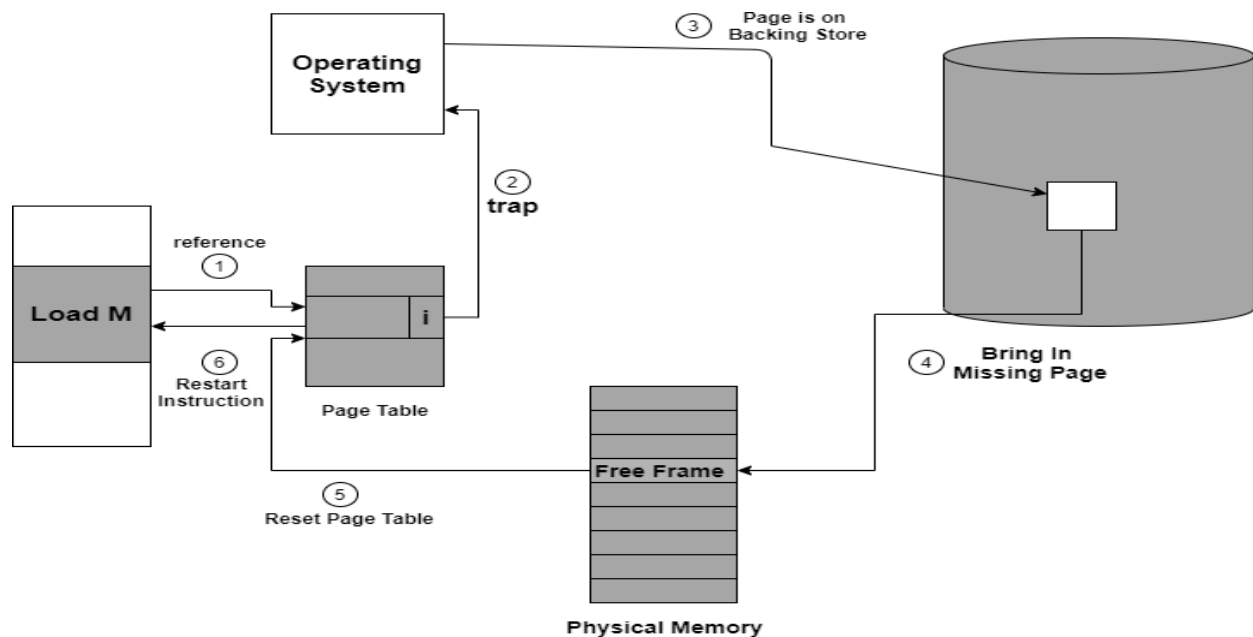
Given below is the simple procedure to handle the page fault:

If you will access a page that is marked as invalid then it also causes a Page Fault. Then the Paging hardware during translating the address through the page table will notice that the invalid bit is set that will cause a trap to the Operating system.

This trap is mainly the result of the failure of the Operating system in order to bring the desired page into memory.

Let us understand the procedure to handle the page fault as shown with the help of the above diagram:

1. First of all, internal table (that is usually the process control block) for this process in order to determine whether the reference was valid or invalid memory access.
2. If the reference is invalid, then we will terminate the process. If the reference is valid, but we have not bought in that page so now we just page it in.
3. Then we **locate the free frame list** in order to find the free frame.
4. Now a disk operation is scheduled in order to read the **desired page into the newly allocated frame**.
5. When the disk is completely read, then the **internal table is modified** that is kept with the process, and the page table that mainly indicates the page is now in memory.
6. Now we will restart the instruction that was interrupted due to the trap. Now the process can access the page as though it had always been in memory.



Page Replacement Algorithms:

As studied in Demand Paging, only certain pages of a process are loaded initially into the memory. This allows us to get more processes into memory at the same time. But what happens when a process requests for more pages and no free memory is available to bring them in. Following steps can be taken to deal with this problem:

1. Put the process in the wait queue, until any other process finishes its execution thereby freeing frames.
2. Or, remove some other process completely from the memory to free frames.
3. Or, find some pages that are not being used right now, move them to the disk to get free frames. This technique is called **Page replacement** and is most commonly used.

Let us discuss all algorithms one by one in the upcoming sections:

1. FIFO Page Replacement Algorithm

It is a very simple way of Page replacement and is referred to as First in First Out. This algorithm mainly replaces the oldest page that has been present in the main memory for the longest time.

Example:

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2	2	4	4	4	0		0	0		7	7	7
	0	0	0		3	3	3	2	2	2		1	1		1	0	0
		1	1		1	0	0	0	3	3		3	2		2	2	1

page frames

Total number of page faults = 15

- This algorithm is implemented by keeping the track of all the pages in the queue.
- As new pages are requested and are swapped in, they are added to the tail of a queue and the page which is at the head becomes the victim.
- This is not an effective way of page replacement but it can be used for small systems.

Consider another reference string: 1,2,3,4,1,2,5,1,2,3,4,5 find number of page faults using three frames and four frames.

For 3 frames, total number of page faults = 9

For 4 frames, total number of page faults = 10

Problem with FIFO page replacement algorithm is “Belady’s anomaly”. For some page reference strings, the page fault rate is increases as the number of allocated frames increases.

Advantages

- This algorithm is simple and easy to use.
- FIFO does not cause more overhead.

Disadvantages

- This algorithm does not make the use of the frequency of **last used time rather** it just replaces the Oldest Page.
- There is an increase in **page faults** as page frames increases.
- The performance of this algorithm is the worst.

2. LIFO Page Replacement Algorithm

This Page Replacement algorithm stands for "Last In First Out". This algorithm works in a similar way to the LIFO principle.

- In this, the newest page is replaced which is arrived at last in the primary memory
- This algorithm makes use of the stack for monitoring all the pages.

3. LRU Page Replacement Algorithm in OS

This algorithm stands for "Least recent used" and this algorithm helps the Operating system to search those pages that are used over a short duration of time frame.

- The page that has not been used for the longest time in the main memory will be selected for replacement.
- This algorithm is easy to implement.
- This algorithm makes use of the counter along with the even-page.

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0			1		1		1		
	0	0	0		0		0	0	3	3			3		0		0		
		1	1		3		3	2	2	2			2		2		7		

page frames

Number of page faults = 12

Advantages of LRU

- It is an efficient technique.
- With this algorithm, it becomes easy to identify the faulty pages that are not needed for along time.
- It helps in Full analysis.

Disadvantages of LRU

- It is expensive and has more complexity.
- There is a need for an additional data structure.

4. Optimal Page Replacement Algorithm

This algorithm mainly replaces the page that will not be used for the longest time in the future.

The practical implementation of this algorithm is not possible.

- Practical implementation is not possible because we cannot predict in advance those pages that will not be used for the longest time in the future.
- This algorithm leads to less number of page faults and thus is the best-known

algorithmAlso, this algorithm can be used to measure the performance of other algorithms.

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		2		2				2				7		
	0	0	0		0		4		0				0				0		
		1	1		3		3		3				1				1		

page frames

Number of page faults = 9

Advantages of OPR

- This algorithm is easy to use.
- This algorithm provides excellent efficiency and is less complex.
- For the best result, the implementation of data structures is very easy

Disadvantages of OPR

- In this algorithm future awareness of the program is needed.
- Practical Implementation is not possible because the operating system is unable to track the future request

5. Random Page Replacement Algorithm

As indicated from the name this algorithm replaces the page randomly. This Algorithm can work like any other page replacement algorithm that is LIFO, FIFO, Optimal, and LRU.

Q. Consider a reference string: 4, 7, 6, 1, 7, 6, 1, 2, 7, 2. the number of frames in the memory is 3. Find out the number of page faults respective to:

1. Optimal Page Replacement Algorithm
2. FIFO Page Replacement Algorithm
3. LRU Page Replacement Algorithm

Optimal Page Replacement Algorithm

Request	4	7	6	1	7	6	1	2	7	2
Frame 3			6	6	6	6	6	2	2	2
Frame 2		7	7	7	7	7	7	7	7	7
Frame 1	4	4	4	1	1	1	1	1	1	1
Miss/Hit	Miss	Miss	Miss	Miss	Hit	Hit	Hit	Miss	Hit	Hit

Number of Page Faults in Optimal Page Replacement Algorithm = 5

LRU Page Replacement Algorithm

Request	4	7	6	1	7	6	1	2	7	2
Frame 3			6	6	6	6	6	6	7	7
Frame 2		7	7	7	7	7	7	2	2	2
Frame 1	4	4	4	1	1	1	1	1	1	1
Miss/Hit	Miss	Miss	Miss	Miss	Hit	Hit	Hit	Miss	Miss	Hit

Number of Page Faults in LRU = 6

FIFO Page Replacement Algorithm

Request	4	7	6	1	7	6	1	2	7	2
Frame 3			6	6	6	6	6	6	7	7
Frame 2		7	7	7	7	7	7	2	2	2
Frame 1	4	4	4	1	1	1	1	1	1	1
Miss/Hit	Miss	Miss	Miss	Miss	Hit	Hit	Hit	Miss	Miss	Hit

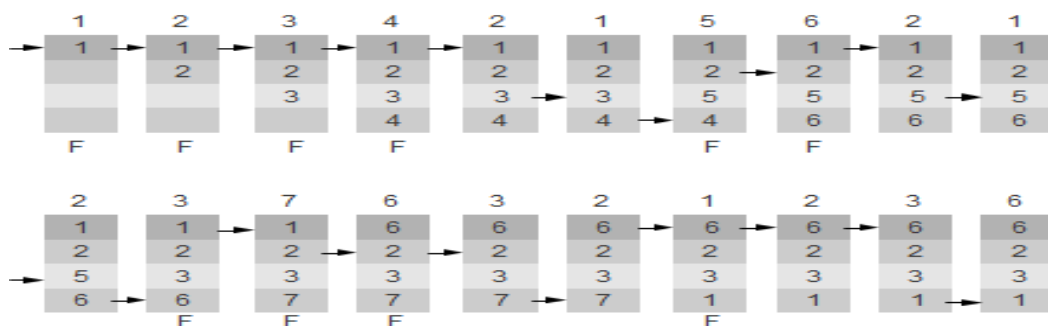
Number of Page Faults in FIFO = 6

Given page reference string:

1,2,3,4,2,1,5,6,2,1,2,3,6

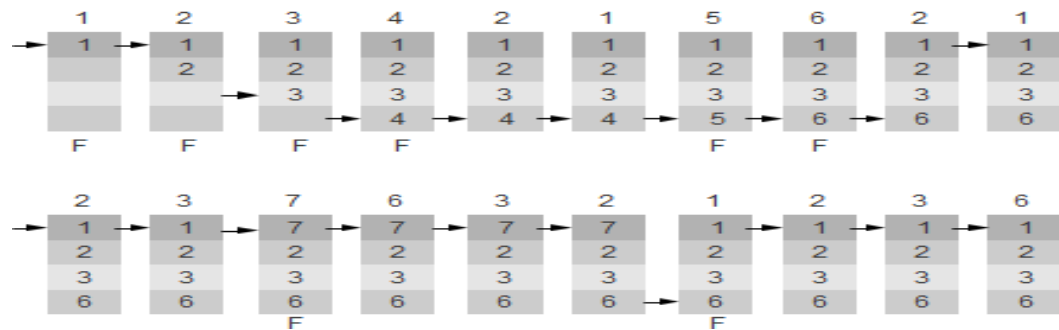
Compare the number of page faults for LRU, FIFO and Optimal page replacement algorithm.

Question 5 : 4 Frames, LRU



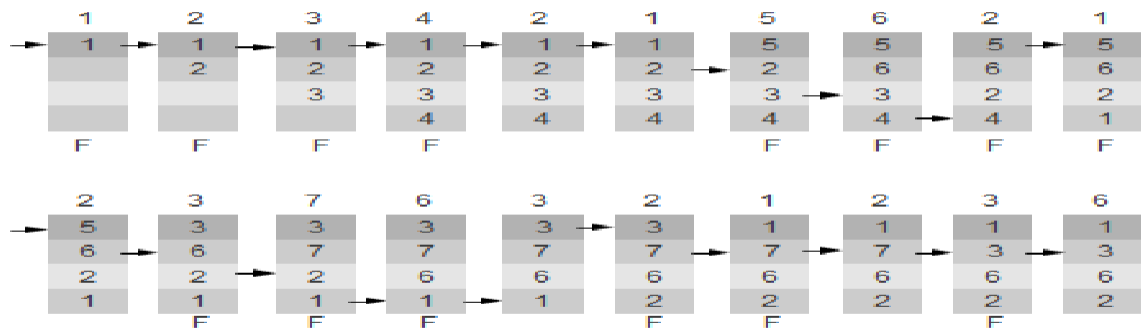
Number of page faults = 10

Question 5 : 4 Frames, OPT



Number of page replacement = 8

Question 5 : 4 Frames, FIFO



ALLOCATION OF FRAMES

- **Minimum number of frames in memory per process**
 - Architecture dependent: All of the pages needed for any possible instruction need to be in memory for this instruction to be executed.
 - Example: the IBM 370 move block operation will need up to 6 pages in memory at the same time in order to execute (two for the instruction, two for the source and two for the destination).
- **Equal allocation**
 - Each process gets the same number of frames. Divide the frames equally between the processes.
- **Proportional allocation**
 - Allocate frames according to size of process.

Consider a system with a 1-KB frame size. If a small student process of 10 KB and an interactive database of 127 KB are the only two processes running in a system with 62 free frames, it does not make much sense to give each process 31 frames. The student process does not need more than 10 frames, so the other 21 are, strictly speaking, wasted. To solve this problem, we can use proportional allocation, in which we allocate available memory to each process according to its size. Let the size of the virtual memory for process P_i be S_i , and define

$$S = \sum s_i.$$

Then, if the total number of available frames is m , we allocate a_i frames to process P_i , where a_i is approximately

$$a_i = s_i / S \times m.$$

For proportional allocation, we can split 62 frames between two processes, one of 10 pages and one of 127 pages, by allocating 4 frames and 57 frames, respectively, since

$$\begin{aligned} 10/137 \times 62 &\approx 4, \text{ and} \\ 127/137 \times 62 &\approx 57. \end{aligned}$$